

This document provides a comprehensive analysis of the existing Excel-based asset management system and a structured roadmap for migrating it to an application.

1. System analysis

1.1 Core calculation engines

Engine 1: Asset condition index (ACI)

Composite score (0–100) per asset from weighted sub-component ratings. For simple assets: ACI = Foundation Rating + Structure Rating. For complex assets like gantries: $ACI = 0.6 \times \text{Foundation} + 0.4 \times \text{Structure}$. Ratings are derived from defect descriptions via lookup tables. Output is classified as Good (≥ 80), Fair (40–79), or Poor (< 40).

Engine 2: Priority index (PI)

$PI = CF_weight \times CF + RF_weight \times RF + FLF_weight \times FLF$, where for gantries: CF = 50%, RF = 30%, FLF = 20%.

- **CF (condition factor):** $IF(ACI \geq 80, 0, (80 - ACI) / 80 \times 100)$ — linear scale from a 101-row lookup table.
- **RF (risk factor):** $10 + ((\text{score} - \text{min}) / (\text{max} - \text{min})) \times 90$ — min-max normalized across 32 asset types.
- **FLF (functional life factor):** Deterioration model per asset type (31 unique equations), e.g. gantry foundation: $\text{consumed} = (ACI - 99.435) / -3.0273$, then $FLF = (\text{consumed} / \text{totalLife}) \times 100$, clamped 0–100.

Engine 3: Defect-to-rating mapping

Text-based defect descriptions are mapped to numeric scores via dropdown validation lists in the ACI calculator. Each asset type has its own defect classification/categorization covering functional and appearance defects. ~~The GANTRY sheet alone contains 41 columns including retroreflectivity (RL values), observation/entrance angles, and color-based sign condition scoring.~~

1.2 Common fields across all asset types

Identity: ASSET_ID, ASSET_NAME, OBJECTID, PMS_MXASSETNUM

Location: ROAD_NUMBER, ROADNAME_EN/AR, ROUTE_ID, DIRECTION, LRS_CHAINAGE, XCOORD, YCOORD, Z_VALUES, COMMUNITY_NAME, AREA_NAME

Condition: FOUNDATION_DEFECTS, STRUCTURE_DEFECTS, FOUNDATION_RATING, STRUCTURE_RATING, ACI, ACI_RATING

Priority: PI, CF, RF, FLF

Survey: DATA_SOURCE, SURVEY_DATE, MANUFACTURER, COMMENTS, ASSET_IMAGE_START/END

2. Technology stack recommendation

Based on the GIS-heavy, calculation-intensive nature of the system, the following stack is recommended:

Component	Recommendation and rationale
Backend	Python (FastAPI) — Deterioration models translate directly from Excel formulas. FastAPI provides automatic OpenAPI docs, async support, and Pydantic validation. Ecosystem (pandas, numpy, geopandas) is unmatched for GIS + engineering.
Database	PostgreSQL + PostGIS — XCOORD/YCOORD fields across 36K records demand spatial queries. PostGIS enables native spatial operations. Schema mirrors Excel sheets with shared lookup tables.
Frontend	React + Leaflet/MapLibre — Interactive dashboards, asset condition heatmaps on E11 roads, filterable tables, and the ACI calculator as a web form replacing Excel dropdowns.
Deployment	Docker containerized web app — Multi-user access (field inspectors → engineers → management), REST API for PMS (Maximo) integration via PMS_MXASSETNUM.

2.1 Why not alternatives?

Option	Drawback for this use case
Java/Spring	Heavier to build, lacks native GIS/numpy ecosystem. Better for enterprise microservices, overkill here.
Node.js	Weak at numeric computation. Deterioration models need numpy-level precision.
Desktop (C#/Electron)	Limits multi-user access. Field inspectors need mobile/tablet access on-site.

3. Migration roadmap

Each phase delivers working functionality. Total estimated timeline: 16 weeks.

Phase 1: Data foundation (weeks 1–3)

Design PostgreSQL/PostGIS schema from the excel sheets. Build ETL pipeline (Python + pandas) to ingest all Excel data. Normalize defect classification into lookup tables. Validate record counts match source files. Create data scripts for repeatable deployment.

Deliverable: Populated database with all records, spatial indices, and automated import scripts.

Phase 2: Calculation engine (weeks 3–6)

Codify the three engines as Python modules: (1) ACI calculator with defect-to-rating mapping per asset type, (2) CF/RF/FLF computation with deterioration equations for all assets, (3) PI (Priority Index) aggregator with configurable weights. Unit-test every formula against Excel-computed values for parity.

Deliverable: Calculation library with 100% formula parity, tested against all existing Excel outputs.

Phase 3: API and core UI (weeks 6–10)

FastAPI REST endpoints for CRUD operations, filtering, and calculation triggers. React frontend with asset list/detail views, ACI calculator form (replacing Excel dropdowns), and map view with GIS overlay of E11 corridor assets. Role-based authentication (inspector vs. engineer vs. admin).

Deliverable: Functional web app replacing core Excel workflows.

Phase 4: Dashboards and reporting (weeks 10–13)

Priority-ranked maintenance dashboard color-coded by ACI rating, with condition heatmaps. Export to PDF/Excel for stakeholder reporting. Historical trend tracking for ACI scores over survey cycles overtime. Batch PI (Priority Index) recalculation when weights are adjusted.

Deliverable: Decision-support dashboards replacing manual Excel analysis.

Phase 5: Integration and scale (weeks 13–16)

Connect to PMS (Maximo) via PMS_MXASSETNUM field for work order generation. Mobile-optimized inspection form for field data collection. Image upload linked to ASSET_IMAGE_START/END fields. Extend to other road corridors beyond E11. API for GIS platform integration (ArcGIS/QGIS).

Deliverable: Production system integrated with existing RTA infrastructure.

4. Risks and mitigation

4.1 Validation strategy

1. **Unit tests:** Every formula (CF, RF, FLF, ACI) tested against hand-calculated values from the Priority Index spreadsheet.
2. **Integration tests:** Full PI calculation for 50 sample gantries compared to Excel output.
3. **Data migration validation:** Row counts, null checks, range checks on every imported table.
4. **User acceptance testing:** Engineers run parallel calculations in both systems for one full survey cycle.

End of document